Exp No: 2	**Multi-Layer Perceptron (MLP) for Iris Classification with Different Hidden Layers and Activation Functions**

AIM:

To develop a multi-layer perceptron for a simple classification task, experiment with different numbers of hidden layers and activation functions, and evaluate the performance of the model using accuracy and loss.

ALGORITHM / PROCEDURE:

1. Import the required libraries and load the Iris dataset (150 samples, 4 features, 3 classes).
2. Split the data into 80% training and 20% testing sets and standardise the features using `StandardScaler`.
3. Define three hidden-layer configurations: `(8,)`, `(16, 8)` and `(32, 16, 8)`.
4. Define three activation functions: *ReLU*, *tanh* and *logistic (sigmoid)*.
5. For every combination, train an `MLPClassifier` with the Adam optimizer (max 1000 iterations).
6. Compute the training loss, test loss (log-loss), training accuracy and test accuracy for each model.
7. Compare all nine models, pick the best one, and display its confusion matrix on the test set.

Fig 1: Structure of the MLP (4 inputs, hidden layers, 3 outputs)

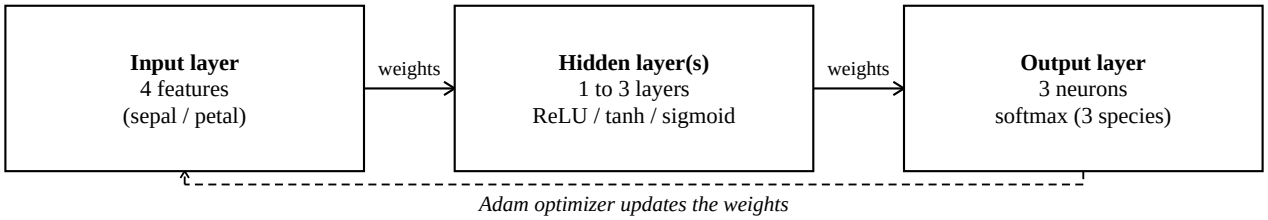
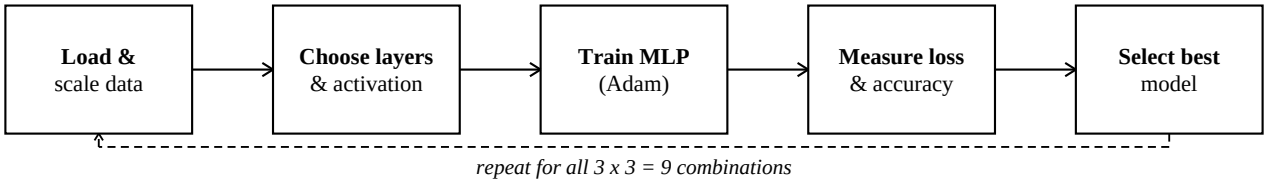


Fig 2: Experiment workflow



Commands / Settings used:

Setting	Values tried	Details
Dataset	Iris	120 training, 30 testing (stratified)
Hidden layers	(8,), (16, 8), (32, 16, 8)	1, 2 and 3 hidden layers
Activation	relu, tanh, logistic	applied in all hidden layers
Optimizer	Adam	max_iter = 1000, random_state = 42
Loss	Cross-entropy (log-loss)	reported on train and test data
Metric	Accuracy	reported on train and test data

PROGRAM:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, log_loss, confusion_matrix
import warnings; warnings.filterwarnings("ignore")

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target)
sc = StandardScaler().fit(X_train)
X_train, X_test = sc.transform(X_train), sc.transform(X_test)

layers = [(8,), (16, 8), (32, 16, 8)]
acts = ["relu", "tanh", "logistic"]

print(f"{'Hidden layers':<14}{'Activation':<11}{'Train loss':<11}"
      f"{'Test loss':<10}{'Train acc':<10}{'Test acc'}")
print("-" * 64)
best = None
for h in layers:
    for a in acts:
        m = MLPClassifier(hidden_layer_sizes=h, activation=a, solver="adam",
                          max_iter=1000, random_state=42).fit(X_train, y_train)
        trl = m.loss_
        tel = log_loss(y_test, m.predict_proba(X_test), labels=[0, 1, 2])
        tra = accuracy_score(y_train, m.predict(X_train))
        tea = accuracy_score(y_test, m.predict(X_test))
        print(f"{str(h):<14}{a:<11}{trl:<11.4f}{tel:<10.4f}{tra*100:<10.2f}{tea*100:.2f}%")
        if best is None or (tea, -tel) > (best[0], -best[1]):
            best = (tea, tel, h, a, m)

tea, tel, h, a, m = best
print(f"\nBest model: hidden layers = {h}, activation = {a}")
print(f"Test accuracy = {tea*100:.2f}%   Test loss = {tel:.4f}")
print("\nConfusion matrix (rows = actual, cols = predicted):")
print(confusion_matrix(y_test, m.predict(X_test)))
```

OUTPUT:

Terminal - MLP on Iris dataset

```
root@ip-10-10-5-10:~# python3 exp2.py
Hidden layers Activation Train loss Test loss Train acc Test acc
----------------------------------------------------------------
(8,)          relu       0.1258     0.1947    96.67     90.00%
(8,)          tanh       0.1489     0.1916    95.83     93.33%
(8,)          logistic   0.2987     0.3358    94.17     86.67%
(16, 8)       relu       0.0514     0.0906    98.33     96.67%
(16, 8)       tanh       0.0676     0.1090    98.33     93.33%
(16, 8)       logistic   0.2124     0.2363    96.67     96.67%
(32, 16, 8)   relu       0.0413     0.0871    98.33     96.67%
(32, 16, 8)   tanh       0.0560     0.0894    98.33     96.67%
(32, 16, 8)   logistic   0.0777     0.0864    98.33     100.00%

Best model: hidden layers = (32, 16, 8), activation = logistic
Test accuracy = 100.00%   Test loss = 0.0864

Confusion matrix (rows = actual, cols = predicted):
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]
```

RESULT:

Thus a multi-layer perceptron was developed for the Iris dataset and evaluated with different numbers of hidden layers and activation functions. Deeper networks gave lower loss than the single 8-neuron layer: the (32, 16, 8) network with logistic activation performed best with 100% test accuracy and a test loss of 0.0864.

Note: with only 30 test samples, one sample changes accuracy by 3.33%, so differences between the top models are small.